

ЛАБОРАТОРНАЯ РАБОТА № 15

Тема: «Создание хранимых процедур, определенных пользователем с использованием СУБД MS SQL Server».

Цель: Научиться создавать хранимые процедуры и вызывать их с помощью сервера MS SQL Server.

Время выполнения: 4 часа

Методическое обеспечение

- 1) Лекции по дисциплине «Базы данных и системы управления базами данных. Раздел 4 – Архитектура клиент-сервер» лекция 2.
- 2) Примеры в лабораторной работе

Задания для самостоятельной работы

Задание 1

Создать *хранимую процедуру*, отвечающую за добавление информации в таблицу (согласно своему варианту).

Используя *SQL-запрос* вызовите созданную процедуру и добавьте в указанную таблицу три новых записи.

Рекомендации к выполнению задания содержатся в Примере 6.

Номер варианта	Таблица
Вариант 1	Поставщик
Вариант 2	Клиент
Вариант 3	Клиент
Вариант 4	Поставщик
Вариант 5	Клиент
Вариант 6	Поставщик
Вариант 7	Клиент
Вариант 8	Клиент
Вариант 9	Клиент

Примечание

Названия таблиц заданы условно. В вашей базе эти таблицы могут называться по-другому.

Задание 2

- 1) Создать хранимую процедуру, со входными параметром, которая изменит данные в таблице. Используйте уже реализованную функцию для вычислений внутри процедуры.
- 2) Используя SQL-запрос вызовите созданную процедуру три раза с различными параметрами.

Рекомендации к выполнению задания содержатся в Примере 7.

Номер варианта	Суть задания
Вариант 1	Создайте хранимую процедуру UpdateWarehouseQuantity, принимающую четыре входных параметра: @DeliverID – номер поставки @PartID – идентификатор детали, @Price – цена за единицу, @NewQuantity – количество. Процедура должна фиксировать факт поставки в таблице ДеталиПоставки и автоматически выполнять перерасчёт общей стоимости поставки в таблице Поставка. Для расчета стоимости использовать функцию.
Вариант 2	Создайте хранимую процедуру, предназначенную для оформления сделки, выполняя автоматический расчёт итоговой суммы. Входные параметры: DEFAULT для даты; @ProductID – идентификатор товара, @NewQuantity – количество.
Вариант 3	Создайте хранимую процедуру, предназначенную для оформления сделки, выполняя автоматический расчёт итоговой суммы с учётом скидки. Входные параметры: DEFAULT для даты; @ProductID – идентификатор товара, @NewQuantity – количество; @NewQuantity – клиент
Вариант 4	Создать хранимую для фиксации поставки. Входные параметры: DEFAULT для даты; Продукт; Количество;

	цена. Реализовать автоматический расчёт общей стоимости: процедура для вставки записи - функция для расчёта суммы.
Вариант 5	Создать хранимую для регистрации оформление сделки. Входные параметры: номер договора, товар, количество, итоговая стоимость должна обновляться и рассчитываться с помощью функции, с учетом розничной / оптовой цены, скидки постоянным клиентам.
Вариант 6	Создать хранимую процедуру, которая регистрирует факт поставки: продукт, количество, цена, автоматический расчёт общей стоимости поставки с использованием функции.
Вариант 7	Создать хранимую для регистрации оформление сделки. Входные параметры: номер договора, товар, количество, итоговая стоимость должна обновляться и рассчитываться с помощью функции, с учетом розничной / оптовой цены, скидки постоянным клиентам.
Вариант 8	Создать хранимую процедуру для добавления позиции в поставку и обновления общей суммы поставки. Входные данные: Номер договора; книга, количество, цена; использовать функцию для расчёта общей суммы.
Вариант 9	Создать хранимую процедуру для оформления аренды. Входные параметры: DEFAULT для даты/времени начала = текущие; точка проката, велосипед, предполагаемая длительность. При регистрации аренды статус

	велосипеда обновляется на «Занят».
--	------------------------------------

Задание 3

Создать *хранимую процедуру, выборки данных*. Процедура должна выводить сведения из связанных таблиц и иметь входной параметр. Используя SQL-запрос вызовите созданную процедуру.

Рекомендации к выполнению задания содержатся в Примере 4.

Номер варианта	Назначение хранимой процедуры
Вариант 1	1. Поставка автозапчастей Создать хранимую процедуру GetSuppliesByPart. Входные параметры: @PartCode NVARCHAR(50) – артикул детали, по которому нужно вывести сведения о поставках. @DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода. <i>Описание: процедура должна выводить поставщика, дату поставки, количество, цену за единицу и общую сумму за указанный период.</i>
Вариант 2	2. Продажа канцтоваров Создать хранимую процедуру GetDealsByClient Входные параметры: @ClientID INT – идентификатор клиента. @DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода. <i>Описание: процедура возвращает список сделок клиента с деталями товаров, количеством, применённой ценой (оптовой/розничной) и итоговой суммой за указанный период.</i>
Вариант 3	3. Продажа IT-товаров Создать хранимую процедуру GetSalesByCategory Входные параметры: @Category NVARCHAR(50) – категория товара. @DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода. <i>Описание: процедура выводит продажи товаров указанной</i>

	категории с указанием клиента, количества, цены и даты сделки за указанный период.
Вариант 4	4. Склад продуктов столовой Создать хранимую процедуру <i>GetProductSupplies</i> Входные параметры: @ProductName NVARCHAR(100) – наименование продукта. @DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода. <i>Описание: процедура</i> показывает поставки продукта: поставщик, дата, количество, цена, общая стоимость за указанный период.
Вариант 5	5. Продажа галантерейных товаров Создать хранимую процедуру <i>GetContractsByClient</i> Входные параметры: @ClientID INT – идентификатор клиента. @MinQuantity INT – минимальное количество товара в сделке (если не указано, выводятся все сделки). <i>Описание: процедура</i> выводит договоры с клиентом, включая товары, количество, цену (оптовую/розничную), скидку и итоговую сумму.
Вариант 6	6. Учёт поступления продуктов в кухню ресторана Создать хранимую процедуру <i>GetRestaurantSupplies</i> Входные параметры: @SupplierID INT – идентификатор поставщика (если не указан, выводятся все поставщики). @DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода. <i>Описание: процедура</i> возвращает список поставок с указанием продукта, количества, цены и общей суммы за указанный период.
Вариант 7	7. Продажа бытовой химии Создать хранимую процедуру <i>GetSalesByVolume</i> Входные параметры: @MinVolume DECIMAL(10,2) – минимальный объём упаковки (если не указан, выводятся все продажи). @DateFrom DATE – начальная дата периода. <i>Описание: процедура</i> выводит продажи товаров с фильтром по минимальному объёму, включая клиента, количество, цену и скидку за указанный период.

Вариант 8	8. Поставка учебной литературы Создать хранимую процедуру GetBookSupplies Входные параметры: @BookTitle NVARCHAR(200) – название учебного издания. @DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода. <i>Описание: процедура выводит поставки указанного издания: поставщик, дата, количество, цена, общая сумма за указанный период.</i>
Вариант 9	9. Прокат велосипедов Создать хранимую процедуру GetRentalsByClient Входные параметры: @ClientID INT – идентификатор клиента. @DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода. <i>Описание: процедура возвращает список прокатов клиента с указанием модели велосипеда, даты начала и окончания аренды, стоимости и залога за указанный период.</i>

Задание 4

Создать *хранимую процедуру*, со входным и выходным параметрами.

Используя SQL-запрос вызовите созданную процедуру.

Рекомендации к выполнению задания содержатся в Примере 5

Номер варианта	Название процедуры	Входные параметры	Выходной параметр	Описание
Вариант 1	GetTotalSuppliesAmount	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма поставок за период.	Выводит общую сумму всех поставок автозапчастей за указанный период.
Вариант 2	GetTotalSalesAmount	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма продаж за период.	Возвращает общую сумму продаж канцтоваров за указанный период.

Вариант 3	GetTotalITSalesAmount	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма продаж за период.	Выводит общую сумму продаж IT-товаров за указанный период.
Вариант 4	GetTotalProductSupplies	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма поставок за период.	Возвращает общую сумму поставок продуктов в столовую за указанный период.
Вариант 5	GetTotalContractsAmount	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма продаж за период.	Выводит общую сумму продаж галантерейных товаров за указанный период.
Вариант 6	GetTotalRestaurantSupplies	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма поставок за период.	Возвращает общую сумму поставок продуктов в ресторан за указанный период.
Вариант 7	GetTotalChemicalsSales	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма продаж за период.	Выводит общую сумму продаж бытовой химии за указанный период.
Вариант 8	GetTotalBookSupplies	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма поставок за период.	Возвращает общую сумму поставок учебной литературы за указанный период.
Вариант 9	GetTotalRentalsAmount	@DateFrom DATE – начальная дата периода. @DateTo DATE – конечная дата периода.	@TotalAmount DECIMAL(18,2) OUTPUT – общая сумма дохода от проката за период.	Выводит общую сумму дохода от аренды велосипедов за указанный период.

Примеры хранимых процедур и их вызова

```
--ПРИМЕР 1
/* ВЫЧИСЛЯЕМАЯ ПРОЦЕДУРА БЕЗ ПАРАМЕТРОВ
   Вычислить количество учащихся */

--CREATE PROCEDURE Count_Student
--AS
--BEGIN
--    Select COUNT(*)
--    from Студенты
--END
```

```
--ПРИМЕР 1
--exec Count_Student
```

```
--ПРИМЕР 2
/* ВЫЧИСЛЯЕМАЯ ПРОЦЕДУРА СО ВХОДНЫМ ПАРАМЕТРОМ
   Вычислить количество учащихся 4 курса*/

--CREATE PROCEDURE Count_Student_kurs
--    @k int
--AS
--BEGIN
--    Select COUNT(*)
--    from Студенты
--    where курс=@k
--END
```

```
--ПРИМЕР 2
--exec Count_Student_kurs @k=4
```



```
--ПРИМЕР 3
/*ПРОЦЕДУРА ВЫБОРКИ ДАННЫХ БЕЗ ПАРАМЕТРА
Ведомость оценок студентов */

--CREATE PROCEDURE Vedomost_ALL
--as
--begin
-- SELECT    Студенты.Фамилия, Студенты.Имя, ПредметОбучения.НазваниеПредмета,
--           ЭкзаменационныеОценки.Оценка, ЭкзаменационныеОценки.ДатаЭкзамена
-- FROM      Студенты INNER JOIN ЭкзаменационныеОценки ON Студенты.КодСтудента = ЭкзаменационныеОценки.КодСтудента
--           INNER JOIN ПредметОбучения ON ЭкзаменационныеОценки.КодПредмета = ПредметОбучения.КодПредмета
--end
```

```
--ПРИМЕР 3
--exec Vedomost_ALL
```

```
--ПРИМЕР 4
/*ПРОЦЕДУРА ВЫБОРКИ ДАННЫХ С ПАРАМЕТРОМ
Ведомость оценок заданного студента */

--CREATE PROCEDURE Vedomost_Student
--@F varchar(50)
--as
--begin
-- SELECT    Студенты.Фамилия, Студенты.Имя, ПредметОбучения.НазваниеПредмета,
--           ЭкзаменационныеОценки.Оценка, ЭкзаменационныеОценки.ДатаЭкзамена
-- FROM      Студенты INNER JOIN ЭкзаменационныеОценки ON Студенты.КодСтудента = ЭкзаменационныеОценки.КодСтудента
--           INNER JOIN ПредметОбучения ON ЭкзаменационныеОценки.КодПредмета = ПредметОбучения.КодПредмета
--           where Студенты.Фамилия like @f
--end
```

```
--ПРИМЕР 4
--exec Vedomost_Student @F='Зязева'
```

```

.ПРИМЕР 5
. ПРОЦЕДУРА С ВХОДНЫМ И ВЫХОДНЫМ ПАРАМЕТРОМ
. Средний балл по дисциплине */

.CREATE PROCEDURE AVG_Subject
@Sbj varchar(50), @p_avg int output
.as
.begin
. SELECT @p_avg=AVG(ЭкзаменационныеОценки.Оценка)
. FROM Студенты INNER JOIN ЭкзаменационныеОценки ON Студенты.КодСтудента = ЭкзаменационныеОценки.КодСтудента
. INNER JOIN ПредметОбучения ON ЭкзаменационныеОценки.КодПредмета = ПредметОбучения.КодПредмета
. where ПредметОбучения.НазваниеПредмета like @Sbj
.end

```

```

--ПРИМЕР 5

--DECLARE @p_avg int

--EXEC AVG_Subject
-- @Sbj = 'Информатика',
-- @p_avg = @p_avg OUTPUT

--SELECT @p_avg as N'@p_avg'

```

```

--ПРИМЕР 6
/* ПРОЦЕДУРА ДОБАВЛЕНИЯ ДАННЫХ СО ВХОДНЫМИ ПАРАМЕТРАМИ*/

--CREATE PROCEDURE INS_UNIV
--@NAME VARCHAR(50), @RAITING INT, @CITY VARCHAR(50)
--AS
--BEGIN
-- INSERT INTO Университеты (НазваниеУниверситета, Рейтинг, Город)
-- Values (@NAME, @RAITING, @CITY)
--END

```

```

--ПРИМЕР 6
--EXEC INS_UNIV @NAME = 'ПГУ', @RAITING =15, @CITY ='Полоцк'

```

```
--ПРИМЕР 7
/* ПРОЦЕДУРА ИЗМЕНЕНИЯ ДАННЫХ СО ВХОДНЫМИ ПАРАМЕТРАМИ*/

--CREATE PROCEDURE ADD_Stipend
--@nadbavka INT, @city CHAR(10)
--AS
--BEGIN
-- UPDATE Студенты SET Стипендия=Стипендия+ @nadbavka
-- WHERE Город= @CITY
--END
```

```
--ПРИМЕР 7
--EXEC ADD_Stipend @nadbavka = 1000, @city = 'Полоцк'
```

```
----ПРИМЕР 8
/*ПРОЦЕДУРА ВЫБОРКИ ИЗ ПРЕДСТАВЛЕНИЯ

--CREATE PROCEDURE Vedomost_Student_VIEW
--@F varchar(50)
--as
--begin
-- SELECT *
-- FROM Vedomost
-- where Vedomost.Фамилия like @f
--end
```

```
--ПРИМЕР 8
--exec Vedomost_Student_VIEW @F='Зязева'
```